CHAPTER 7

Standard Types

We've been having fun implementing programs using arrays, hashes, and procs, but we haven't covered the most basic types in Ruby: numbers, strings, and ranges. Let's spend a few pages on these basic building blocks now.

Numbers

Ruby supports integers, floating-point, rational, and complex numbers. Integers can be any length (up to a maximum determined by the amount of free memory on your system), and are of type Integer.

Integers are assumed to be decimal base 10, but, you can write integers using a leading sign as an optional base indicator—0 for octal, 0x for hex, or 0b for binary (and 0d for decimal)—followed by a string of digits in the appropriate base.

Underscore characters are ignored in the digit string, you'll see them used in place of commas in larger numbers.

```
123456                  => 123456   # base 10
0d123456                => 123456   # base 10
123_456                 => 123456   # underscore ignored
-543                    => -543     # negative number
0xaabb                  => 43707    # hexadecimal
0377                    => 255      # octal
-0b10_1010              => -42      # binary (negated)
123_456_789_123_456_789 => 123456789123456789
```

A numeric literal with a decimal point and/or an exponent is turned into a Float corresponding to double-width floating point numbers in the underlying system. You must both precede and follow the decimal point with a digit—if you write 1.0e3 as 1.e3, Ruby will try to invoke the method e3 on the object 1.

The standard Ruby library contains the BigDecimal class, which is Ruby's high-precision decimal class. When you require BigDecimal, in addition to the class itself, the Kernel module gets a BigDecimal method for converting strings or numbers to BigDecimal instances.

```
require "bigdecimal"
x = BigDecimal("3.14")
y = BigDecimal("4.13")
x + y     # => 0.727e1
```